

> TECHNICAL REPORT // PWM PROJECT // V1.0

RELAZIONE TECNICA_

**BURGER.SH — PIATTAFORMA WEB PER
ORDINAZIONI ONLINE**

STUDENTE	Joel Stephan
MATRICOLA	80311A
CORSO	Programmazione Web e Mobile
ANNO	2025/2026
UNIVERSITÀ	Università degli Studi di Milano

[SYSTEM READY]

BURGER.SH – RELAZIONE TECNICA

PROGETTO PER IL CORSO DI PROGRAMMAZIONE WEB E MOBILE

Studente: Joel Stephan Matricola: 80311A Anno accademico: 2025/2026 Università:
Università degli Studi di Milano

INDICE

1. Frontespizio
2. Indice
3. Introduzione
4. Analisi dei Requisiti
5. Modellazione del Dominio
6. Architettura del Sistema
7. Progettazione delle API
8. Implementazione Frontend
9. Scelte Progettuali e Motivazioni
10. Scenari di Demo
11. Conclusioni
12. Bibliografia e Sitografia

1. FRONTESPIZIO

Questo documento costituisce la relazione tecnica del progetto **Burger.sh**, realizzato come lavoro finale per il corso di Programmazione Web e Mobile dell'Università degli Studi di Milano, aa. 2025/2026.

Il progetto implementa una piattaforma web per la gestione di ordinazioni online in una catena di ristoranti fast food, con supporto per tre ruoli utente (cliente, manager, amministratore), gestione del menu, elaborazione ordini, calcolo consegne a domicilio tramite API di geolocalizzazione e dashboard statistiche.

2. INDICE

(L'indice e' riportato all'inizio del documento al punto 2.)

3. INTRODUZIONE

Ho sviluppato Burger.sh come piattaforma web per la gestione di ordinazioni online all'interno di una catena di ristoranti fast food. L'applicazione nasce dalla volontà di digitalizzare il processo di ordinazione, permettendo ai clienti di consultare i menu, selezionare i piatti, effettuare ordini con ritiro in sede o consegna a domicilio, e seguire lo stato della propria ordinazione in tempo reale. Parallelamente, i manager di ogni filiale possono gestire il proprio menu, visualizzare e aggiornare gli ordini ricevuti e consultare statistiche di vendita.

Ho articolato il sistema in quattro macro-scenari principali. Il primo riguarda la gestione del profilo utente: registrazione, login, modifica dei dati personali, eliminazione dell'account e gestione delle preferenze (tipologie alimentari, allergie). Il secondo è la gestione del ristorante: creazione e amministrazione della filiale da parte dell'amministratore, gestione del menu con aggiunta, modifica ed eliminazione dei piatti, organizzazione degli ingredienti. Il terzo scenario è la gestione degli ordini: dal carrello in bozza alla conferma, passando per la selezione dei piatti, la scelta della modalità di completamento e il tracciamento dello stato dell'ordine. Il quarto è la gestione delle consegne: calcolo della distanza tra ristorante e indirizzo del cliente, determinazione del costo di consegna, e conferma della ricezione da parte del cliente.

Lo stack che ho scelto è composto da Preact con Vite come bundler e Tailwind CSS per lo styling nel frontend. Ho deciso di usare Preact per evitare di complicare il progetto con la gestione manuale del DOM e per migliorare la manutenibilità del codice frontend attraverso un'architettura a componenti. Vite l'ho scelto per semplificare lo sviluppo grazie al suo hot module replacement e alla velocità di build. Il backend è sviluppato con Node.js ed Express v5, con MongoDB come database NoSQL accessibile tramite Mongoose v9. Le API REST sono documentate con Swagger/OpenAPI 3.0.3. Per il calcolo delle distanze e dei costi di consegna, ho integrato le API di OpenStreetMap (Nominatim per il geocoding e OSRM per il routing). L'autenticazione è gestita tramite token JWT, con hashing delle password mediante bcrypt.

La versione Light del progetto implementa le funzionalità core: registrazione e login per cliente e manager, CRUD del profilo utente, CRUD del ristorante e del menu, ricerca piatti per tipologia e nome, visualizzazione dettagli piatto. La versione Full aggiunge la ricerca per ingrediente e allergie, la gestione completa degli ordini con carrello, la scelta tra ritiro e consegna, il calcolo della distanza tramite OpenStreetMap, la conferma di consegna, le statistiche per ristorante e lo storico acquisti. Ho inoltre aggiunto funzionalità estese non richieste dalla traccia originale, come il ruolo amministratore, la gestione dei metodi di pagamento e la gestione degli ingredienti.

4. ANALISI DEI REQUISITI

Di seguito e' riportata la tabella dei requisiti con il loro stato di implementazione nella versione Full.

ID	REQUISITO	STATO (LIGHT/ FULL)
R01	Registrazione e login per cliente e ristoratore	Light / Full
R02	CRUD profilo utente (cliente/ristoratore)	Light / Full
R03	CRUD ristorante e menu (piatti da meal.json + piatti custom)	Light / Full
R04	Ricerca piatti per tipologia, nome, prezzo	Light / Full
R05	Visualizzazione dettagli piatto (ingredienti, foto, info)	Light / Full
R06	Visualizzazione info piatti, clienti, ristoratori, acquisti	Full
R07	Ricerca ristoranti per luogo e nome	Full
R08	Ricerca ristorante per piatto	Full
R09	Ricerca piatti per ingredienti	Full
R10	Ricerca piatti per allergie	Full
R11	Creazione ordine con più piatti	Full
R12	Scelta ritiro in sede o consegna a domicilio	Full
R13	Flusso stati ordine (ordinato → in preparazione → in consegna → consegnato)	Full
R14	Calcolo distanza e costo consegna tramite API OpenStreetMap	Full
R15	Gestione consegne (conferma consegna da parte del cliente)	Full
R16	Statistiche per ristorante (ordini, piatti più venduti, ecc.)	Full
R17	Storico acquisti cliente (presenti e passati)	Full
R18	Ruolo amministratore con gestione filiali e manager	Full
R19	Gestione metodi di pagamento del cliente	Full
R20	Gestione ingredienti (pubblici e privati)	Full
R21	Dashboard manager con ordini in coda e statistiche	Full

Ogni requisito e' stato tradotto in endpoint API REST e componenti frontend. La registrazione e il login (R01) corrispondono agli endpoint POST /auth/register e POST /auth/login, che restituiscono un token JWT. Il profilo utente (R02) e' gestito dagli endpoint GET/PUT/DELETE /users/me. La gestione del ristorante (R03) coinvolge gli endpoint su /restaurants e /dishes, con supporto per piatti standard (caricati da meal.json) e piatti custom. La ricerca (R04-R10) e' implementata tramite parametri di query sulle rotte /dishes e /restaurants. Gli ordini (R11-R13) seguono un flusso basato su un carrello in bozza (collezione orders con status "draft") che viene confermato tramite POST /cart/confirm. La consegna a domicilio (R14-R15) utilizza il servizio deliveryService che chiama le API OpenStreetMap per geocodificare l'indirizzo e calcolare la distanza. Le statistiche (R16) sono rese dall'endpoint GET /orders/restaurant/:id/dashboard. Lo storico ordini (R17) e' disponibile su GET /orders/user. I requisiti aggiuntivi (R18-R21) sono stati implementati come estensioni progettuali per arricchire la piattaforma.

5. MODELLAZIONE DEL DOMINIO

5.1 ENTITÀ PRINCIPALI

Le entità principali del dominio che ho modellato sono le seguenti:

- **User:** rappresenta tutti gli utenti del sistema, con tre ruoli possibili: customer (cliente), manager (gestore di filiale), admin (amministratore centrale). Contiene i dati anagrafici, le credenziali di accesso, l'indirizzo e le preferenze alimentari. Per i manager e' presente il campo managerStatus che ne traccia lo stato di approvazione.
- **Restaurant:** rappresenta una filiale della catena. Contiene nome, indirizzo, città, telefono, partita IVA e il riferimento al manager proprietario. Include inoltre coordinate geografiche (lat/lng) geocodificate al bisogno.
- **Dish:** rappresenta un piatto del menu. Può essere standard (isCustom: false, presente in tutti i ristoranti) o custom (isCustom: true, specifico di una filiale). Contiene nome, tipologia, prezzo, url della foto e riferimenti agli ingredienti.
- **Ingredient:** rappresenta un ingrediente utilizzato nei piatti. Può essere pubblico (restaurantId: null) o privato di una filiale. Contiene nome e lista di allergeni associati.
- **Order:** rappresenta un ordine o un carrello in bozza. Contiene il riferimento al cliente, al ristorante, le righe ordine (orderItems), lo stato corrente, la modalità (pickup/delivery), l'importo totale, il codice ordine e, per le consegne, il sottodocumento delivery con indirizzo, distanza e costo.
- **PaymentMethod:** rappresenta un metodo di pagamento associato a un cliente. Contiene tipo (card/cash), etichetta, dettagli e flag del metodo predefinito.

5.2 SCHEMA COLLEZIONI MONGODB

Collection: **users**

- `_id`: ObjectId
- `name`: String (nome, required)
- `surname`: String (cognome, required)
- `email`: String (unique, lowercase, required)
- `passwordHash`: String (hash bcrypt, required)
- `role`: String (enum: "customer" | "manager" | "admin", required)
- `address`: Object (street, city, zip)
- `managerStatus`: String (enum: "pending" | "approved" | "rejected", presente solo per role="manager")
- `restaurantId`: ObjectId (ref: Restaurant, default null)
- `preferences`: Array of String (enum da costante)
- `mustChangePassword`: Boolean (default false)
- `setupCompleted`: Boolean (default false)
- `createdAt`: Date (automatic)
- `updatedAt`: Date (automatic)

Collection: **restaurants**

- `_id`: ObjectId
- `name`: String (required)
- `address`: String (required)
- `city`: String (required)
- `zip`: String
- `phone`: String (required)
- `vatNumber`: String (required)
- `managerId`: ObjectId (ref: User, required)
- `location`: Object (lat, lng, geocoded on demand)
- `createdAt`: Date (automatic)
- `updatedAt`: Date (automatic)

Collection: dishes

- `_id`: ObjectId
- `name`: String (required)
- `type`: String (required, es. "burger", "drink", "side")
- `price`: Number (required, min 0)
- `photoUrl`: String
- `ingredientIds`: Array of ObjectId (ref: Ingredient)
- `isCustom`: Boolean (default false)
- `restaurantId`: ObjectId (ref: Restaurant, default null, populated when `isCustom=true`)
- `createdAt`: Date (automatic)
- `updatedAt`: Date (automatic)

Collection: ingredients

- `_id`: ObjectId
- `name`: String (required)
- `allergens`: Array of String
- `restaurantId`: ObjectId (ref: Restaurant, default null per ingredienti pubblici)
- `managerId`: ObjectId (ref: User, default null, usato durante il wizard di setup)
- `createdAt`: Date (automatic)
- `updatedAt`: Date (automatic)

Collection: orders

- `_id`: ObjectId
- `customerId`: ObjectId (ref: User, required)
- `restaurantId`: ObjectId (ref: Restaurant, required)
- `orderItems`: Array of {dishId, quantity, unitPrice} (embedded)
- `status`: String (enum: "draft" | "ordered" | "preparing" | "ready" | "on_delivery" | "delivered", required, default "ordered")
- `mode`: String (enum: "pickup" | "delivery", required when status !== "draft")
- `totalAmount`: Number (required, default 0)
- `orderCode`: String (unique, required, es. "FF-A1B2C3")
- `delivery`: Object (address, distanceKm, deliveryFee, optional, assente when mode="pickup")
- `createdAt`: Date (automatic)
- `updatedAt`: Date (automatic)
- Index: unique_draft_per_customer (partial, on customerId + status where status="draft")

Collection: paymentMethods

- `_id`: ObjectId
- `customerId`: ObjectId (ref: User, required)
- `type`: String (enum: "card" | "cash", required)
- `label`: String
- `details`: String (solo ultime 4 cifre per le carte)
- `isDefault`: Boolean (default false)
- `createdAt`: Date (automatic)
- `updatedAt`: Date (automatic)

5.3 DIAGRAMMA CONCETTUALE

Il diagramma ER del dominio che ho progettato prevede le seguenti relazioni principali. Un utente può avere ruolo customer, manager o admin. Un utente con ruolo manager può essere associato a una singola filiale tramite il campo `restaurantId`. Un ristorante ha un unico manager proprietario (`managerId`) e può avere zero o più piatti nel menu. I piatti possono essere standard (condivisi tra tutte le filiali) o custom (appartenenti a una singola filiale). Un ordine appartiene a un cliente e a un ristorante, e contiene una o più righe ordine, ciascuna delle quali fa riferimento a un piatto e ne memorizza quantità e prezzo unitario al momento dell'acquisto. Un ordine può avere modalità pickup o delivery. Nel caso di delivery, e' presente un sottodocumento `delivery` con indirizzo, distanza e costo. Una delivery non e' un'entità separata nel database, ma e' parte integrante dell'ordine. Gli ingredienti sono un'entità indipendente, associabili a zero o più piatti e, nel caso di ingredienti privati, a una specifica filiale. I metodi di pagamento sono associati a un singolo cliente.

Il diagramma completo e' disponibile come file Draw.io nella cartella docs/ita/diagrams/fastfood-domain-model.drawio.

6. ARCHITETTURA DEL SISTEMA

6.1 SCHEMA ARCHITETTURALE

Ho articolato il sistema in tre livelli principali.

Il **frontend** e' una Single Page Application (SPA) costruita con Preact (libreria UI dichiarativa simile a React, ~3kb) e Vite (bundler moderno). Lo styling e' gestito tramite Tailwind CSS v4 con un design system custom ispirato all'estetica retro-cyberpunk (terminal/CRT). La navigazione e' gestita da un router custom basato su match di pattern del path URL, senza librerie esterne. Le chiamate API sono effettuate tramite fetch con wrapper personalizzato che gestisce il token JWT, il retry su errori di rete e il redirect in caso di session scaduta. Lo stato dell'applicazione e' gestito tramite store Preact (context-based) separati per autenticazione, ordini e ristoranti.

Il **backend** e' un server HTTP costruito con Node.js ed Express v5. La struttura e' modulare con controller, modelli Mongoose, route, middleware e validation schema distribuiti in cartelle separate. I middleware includono autenticazione JWT, controllo ruoli, validazione delle request body e query, paginazione, gestione errori e rate limiting sulle rotte di setup. L'applicazione usa helmet per la sicurezza HTTP, morgan per il logging e express-rate-limit per proteggere le rotte critiche. L'integrazione con MongoDB e' gestita da Mongoose v9 con modelli mongoose definiti per ciascuna collezione.

Il **database** e' MongoDB, un database NoSQL orientato a documenti. La modellazione segue uno stile ibrido: embedding per dati strettamente legati al ciclo di vita del documento padre (righe ordine e consegna dentro Order), referencing per dati riutilizzabili o condivisi (utenti, ristoranti, piatti, ingredienti).

Un **servizio esterno** e' rappresentato dalle API di OpenStreetMap, utilizzate per il geocoding degli indirizzi (Nominatim) e per il calcolo delle distanze stradali (OSRM). Il backend memorizza le coordinate geocodificate dei ristoranti nella collezione restaurants per evitare richieste ripetute al servizio esterno.

6.2 FLUSSO RICHIESTE REST

Quando un utente autenticato effettua una richiesta, ad esempio POST /cart/items per aggiungere un piatto al carrello, il flusso e' il seguente.

1. Il frontend richiama `api.post('/cart/items', {restaurantId, dishId, quantity})` dal servizio `orderService`. La funzione `fetchWithAuth` inserisce automaticamente l'header `Authorization: Bearer <token>` letto da `localStorage`.
2. La richiesta arriva al server Express, che passa attraverso i middleware globalmente installati (`helmet`, `morgan`, `cors`).
3. Il router monta `cartRoutes.js`, che applica il middleware `authMiddleware` per verificare il token JWT. Se il token è valido, il controller dell'utente autenticato è aggiunto a `req.user`.
4. Il middleware `validate` applica lo schema Joi definito in `cartValidation.js` per validare il body della richiesta.
5. Il controller `cartController.addItem` riceve la richiesta, cerca o crea il carrello in bozza per quel cliente, aggiunge o aggiorna la riga ordine, e salva su MongoDB tramite il modello `Order`.
6. Il modello `Order` esegue la query di insert/update asincrona su MongoDB.
7. Il controller restituisce una risposta JSON con lo stato dell'ordine aggiornato.
8. Il frontend riceve la risposta e aggiorna lo store locale (`orderStore`) tramite `setCart`, che provvede a riflettere i cambiamenti nell'interfaccia utente grazie al meccanismo reattivo di `Preact`.

Per le rotte pubbliche (es. `GET /restaurants`, `GET /dishes`), i passaggi 2-4 sono identici, ma il middleware `authMiddleware` non è applicato, quindi la richiesta è svolta senza verifica del token.

7. PROGETTAZIONE DELLE API

Le API REST del progetto sono state progettate seguendo i principi RESTful, con un focus sulla chiarezza dei nomi delle risorse e sulla separazione dei concerns tra le diverse entità. La documentazione completa e' disponibile in formato Swagger/OpenAPI 3.0.3 nel file `backend/swagger/openapi.yaml`, accessibile interattivamente all'indirizzo `http://localhost:3000/api-docs` in ambiente di sviluppo.

Le risorse principali che ho implementato sono:

Autenticazione: POST `/auth/register` e POST `/auth/login` per la registrazione e il login. Entrambi restituiscono un token JWT valido per 3 ore.

Utenti: GET/PUT/DELETE `/users/me` per la gestione del profilo dell'utente autenticato. Include anche la gestione dei metodi di pagamento con GET/POST/PUT/DELETE `/users/me/payment-methods`.

Ristoranti: GET `/restaurants` (pubblico, con filtri name, city, dishName), GET `/restaurants/:id` (pubblico), POST `/restaurants` (solo admin), POST `/restaurants/first` (manager approvato che crea la prima filiale), PUT/DELETE `/restaurants/:id` (admin/manager).

Piatti: GET `/dishes` (pubblico, con filtri name, type, minPrice, maxPrice, ingredient, allergen), GET `/dishes/restaurant/:id` (pubblico), GET `/dishes/:id` (pubblico), POST/PUT/DELETE `/dishes` (admin o manager per piatti custom).

Ingredienti: GET/POST/PUT/DELETE `/ingredients` (solo manager).

Carrello: GET `/cart`, POST/PATCH/DELETE `/cart/items`, DELETE `/cart`, POST `/cart/confirm`, POST `/cart/delivery-estimate`. Il carrello e' un ordine in stato "draft" che puo' essere modificato liberamente prima della conferma.

Ordini: POST `/orders` (ordine diretto), GET `/orders/user` (storico cliente), GET `/orders/restaurant/:id` (ordini filiale), GET `/orders/restaurant/:id/dashboard` (statistiche), GET `/orders/:id` (dettaglio), PATCH `/orders/:id/status` (avanzamento stato), PATCH `/orders/:id/confirm-delivery` (conferma ricezione).

Admin: GET/POST/PUT/DELETE `/admin/users`, GET `/admin/stats`.

Setup: GET `/setup/status`, POST `/setup/request-pin`, POST `/setup` (tutti rate-limited).

Di seguito tre esempi completi di request body e response JSON per endpoint significativi.

Esempio 1: POST `/auth/register` (registrazione cliente)

Request body:

```
{
  "name": "Mario",
  "surname": "Rossi",
  "email": "mario.rossi@example.com",
  "password": "password123",
  "role": "customer",
  "address": {
    "street": "Via Roma 1",
    "city": "Milano",
    "zip": "20100"
  },
  "preferences": ["vegetariano", "senza_glutine"]
}
```

Response (201):

```
{
  "success": true,
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "user": {
      "id": "64b7a0f9a1234567890abcd0",
      "name": "Mario",
      "surname": "Rossi",
      "email": "mario.rossi@example.com",
      "role": "customer",
      "address": {
        "street": "Via Roma 1",
        "city": "Milano",
        "zip": "20100"
      },
      "preferences": ["vegetariano", "senza_glutine"],
      "createdAt": "2025-06-15T10:30:00.000Z"
    }
  }
}
```

Esempio 2: POST /cart/confirm (conferma ordine con consegna a domicilio)

Request body:

```
{
  "mode": "delivery",
  "delivery": {
    "address": "Via Torino 5, Milano"
  }
}
```

Response (200):

```
{
  "success": true,
  "data": {
    "_id": "64b7a0f9a1234567890def01",
    "customerId": "64b7a0f9a1234567890abcd0",
    "restaurantId": "64b7a0f9a1234567890abc1",
    "orderItems": [
      {
        "dishId": "64b7a0f9a1234567890abc2",
        "quantity": 2,
        "unitPrice": 6.50
      },
      {
        "dishId": "64b7a0f9a1234567890abc3",
        "quantity": 1,
        "unitPrice": 3.00
      }
    ],
    "status": "ordered",
    "mode": "delivery",
    "totalAmount": 15.50,
    "orderCode": "FF-A1B2C3",
    "delivery": {
      "address": "Via Torino 5, Milano",
      "distanceKm": 3.2,
      "deliveryFee": 2.50
    },
    "createdAt": "2025-06-15T12:45:00.000Z"
  }
}
```

Esempio 3: PATCH /orders/:id/status (avanzamento stato ordine da parte del manager)

Request body:

```
{
  "status": "preparing"
}
```

Response (200):

```
{
  "success": true,
  "data": {
    "_id": "64b7a0f9a1234567890def01",
    "customerId": "64b7a0f9a1234567890abcd0",
    "restaurantId": "64b7a0f9a1234567890abc1",
    "orderItems": [
      {
        "dishId": "64b7a0f9a1234567890abc2",
        "quantity": 2,
        "unitPrice": 6.50
      }
    ],
    "status": "preparing",
    "mode": "pickup",
    "totalAmount": 13.00,
    "orderCode": "FF-A1B2C3",
    "createdAt": "2025-06-15T12:45:00.000Z",
    "updatedAt": "2025-06-15T12:50:00.000Z"
  }
}
```

8. IMPLEMENTAZIONE FRONTEND

8.1 PAGINE PRINCIPALI

Il frontend è organizzato come SPA con routing basato su path. Di seguito le pagine principali che ho implementato.

Auth page (/auth): Gestisce il flusso di login e registrazione. Presenta un form con campi per nome, cognome, email, password e ruolo. Dopo la registrazione, se l'utente è un manager, l'account resta in stato pending fino all'approvazione di un admin. Dopo il login, il token JWT è salvato in localStorage e lo store authStore è aggiornato.

HomePage (/): Landing page con stile terminal/CRT. Presenta un hero section con il messaggio di benvenuto e due bottoni: uno per accedere al menu (navigate a /orders) e uno per accedere al terminale di autenticazione (navigate a /auth).

RestaurantSelectPage (/restaurants, /orders): Pagina che mostra l'elenco dei ristoranti disponibili con possibilità di filtrare per nome e città. Permette di selezionare un ristorante per vedere il menu.

OrderMenuPage (/orders/menu/:restaurantId): Visualizza il menu del ristorante selezionato con i piatti disponibili, ordinati per tipologia. Per ogni piatto è possibile vedere nome, prezzo, foto e ingredienti. I piatti possono essere aggiunti al carrello.

OrderPaymentPage (/orders/payment): Pagina di riepilogo ordine dove il cliente conferma la modalità di completamento (ritiro in sede o consegna a domicilio). Per la consegna, è possibile inserire l'indirizzo e ottenere una stima della distanza e del costo tramite l'endpoint /cart/delivery-estimate.

OrderDetailPage (/orders/:orderId): Mostra i dettagli di un ordine con stato corrente, righe ordine, totale e, per le consegne, indirizzo e costo di consegna. Il cliente può confermare la ricezione se l'ordine è nello stato on_delivery.

CustomerDashboard (/dashboard): Dashboard del cliente con accesso allo storico ordini (passati e attuali), gestione del profilo e metodi di pagamento.

ManagerOverview (/dashboard/manager): Dashboard principale del manager con vista sugli ordini in arrivo, ordini in preparazione e ordini completati. Link alle sottopagine menu, ingredienti e gestione ristorante.

ManagerOrders (/manager/orders): Pagina che mostra gli ordini della filiale con possibilità di filtrare per stato. Il manager può avanzare lo stato di ciascun ordine tramite un menu a tendina.

ManagerMenu (/manager/menu): Pagina per la gestione del menu. Mostra i piatti standard (sola lettura) e i piatti custom (modificabili). Permette di aggiungere, modificare ed eliminare piatti custom, con selezione degli ingredienti disponibili.

ManagerIngredients (/manager/ingredients): Pagina per la gestione degli ingredienti. Permette di creare, modificare ed eliminare ingredienti pubblici e privati (associati alla filiale).

ManagerRestaurant (/manager/restaurant): Pagina per la gestione delle informazioni della filiale (nome, indirizzo, telefono, partita IVA).

AdminOverview (/dashboard/admin): Dashboard amministrativa con accesso alle pagine Utenti, Filiali e Statistiche.

AdminUsers (/admin/users): Elenco di tutti gli utenti con possibilità di filtrare per ruolo e stato manager. Permette di approvare o rifiutare i manager pending.

AdminBranches (/admin/branches): Gestione delle filiali: creazione, modifica e chiusura.

AdminStats (/admin/stats): Statistiche aggregate della piattaforma: utenti per ruolo, totale filiali, ordini totali e per stato.

ProfilePage (/profile): Pagina per la gestione del profilo utente: visualizzazione e modifica dei dati personali, indirizzo, preferenze e metodi di pagamento.

SetupPage (/setup): Pagina per il setup iniziale dell'applicazione. Richiede un PIN inviato via email e permette di creare il primo account admin.

CreateFirstRestaurantPage (/manager/first-restaurant): Wizard per la creazione della prima filiale da parte di un manager approvato. Guida l'utente attraverso l'inserimento dei dati della filiale e il caricamento del menu base.

8.2 PATTERN CSS/JS

Il frontend segue questi pattern architetturali.

La separazione tra struttura e presentazione è garantita dall'uso di Preact con template htm. I componenti UI sono definiti come funzioni che ritornano template htm, con styling gestito tramite classi Tailwind CSS e CSS custom definito in tokens.css e terminal.css. Non è presente un file CSS monolitico: lo stile è distribuito tra utility Tailwind (layout, spacing, tipografia) e regole custom (effetti CRT, animazioni orbitali, colori semantici).

La logica JavaScript è organizzata in servizi separati per dominio (api.js, authService.js, orderService.js, restaurantService.js, paymentService.js, setupService.js, systemService.js). Ogni servizio è un oggetto con metodi asincroni che incapsulano le chiamate API. Il wrapper api.js gestisce automaticamente il token JWT leggendolo da localStorage, attachinglo all'header Authorization, gestendo errori 401 con redirect al login e retry automatico su errori di rete.

Lo stato dell'applicazione è gestito tramite store Preact basati su context e useState. I principali store sono: authStore (utente autenticato, loading, login/logout), orderStore (carrello, ordini), restaurantStore (ristoranti, piatti) e uiStore (stato UI globale come sidebar, notifiche). Gli store sono forniti come provider a livello di root dell'applicazione (main.jsx) e consumati dai componenti tramite il hook useContext.

La gestione del token JWT avviene tramite localStorage. Al login/registrazione, il token è salvato in localStorage e nella variabile locale authToken del modulo api.js. Ad ogni richiesta API, il token è letto e aggiunto come header Bearer. In caso di risposta 401, il token è cancellato e l'utente è reindirizzato alla pagina di login.

L'aggiornamento dinamico del DOM è gestito in modo dichiarativo da Preact: quando lo store è aggiornato (attraverso setter negli store), i componenti che consumano quello store vengono riconnessi e il DOM è ricreato automaticamente. Non è usato innerHTML o createElement manualmente.

8.3 RESPONSIVE DESIGN

Il sito e' completamente responsive grazie a Tailwind CSS con il suo sistema di utility responsive (breakpoint sm, md, lg, xl). Il layout utilizza flexbox e grid per adattarsi a diverse dimensioni dello schermo. La navigazione laterale (sidebar) e' collassabile su mobile. I component card dei ristoranti e dei piatti usano griglie responsive che passano da una colonna su mobile a due o tre colonne su desktop. I form di login, registrazione e gestione ordine sono centrati e adattano le dimensioni dei campi in base al breakpoint. I colori e i font sono definiti tramite CSS variables che garantiscono coerenza visiva su tutti i dispositivi.

9. SCELTE PROGETTUALI E MOTIVAZIONI

JWT vs Sessioni. Ho scelto l'autenticazione basata su token JWT (JSON Web Token) anziché sessioni server-side perché l'architettura del sistema è stateless: il backend non deve memorizzare lo stato della sessione di alcun utente. Questo è particolarmente importante per un servizio API REST che potrebbe essere scalato orizzontalmente, con più istanze del server che non condividono memoria. Il token JWT contiene le informazioni essenziali dell'utente (id, ruolo) ed è firmato con una chiave segreta, quindi il backend può verificarne l'autenticità senza interpellare il database ad ogni richiesta. I token hanno una durata di 3 ore, sufficiente per una sessione di lavoro ma non troppo lunga da compromettere la sicurezza in caso di furto. Il vantaggio rispetto alle sessioni tradizionali è la semplicità e la scalabilità. Lo svantaggio è che un token rubato rimane valido fino alla scadenza, mentre una sessione server-side può essere invalidata immediatamente.

MongoDB vs SQL. La scelta di MongoDB è caduta sul fatto che il modello dei dati del progetto è naturalmente orientato ai documenti: un ordine contiene un array di righe ordine, un ristorante ha un array di piatti, un utente ha un array di preferenze e metodi di pagamento. Con un database relazionale, queste strutture avrebbero richiesto tabelle aggiuntive con chiavi esterne e join complessi. Con MongoDB, i dati correlati possono essere embedded direttamente nel documento principale (come orderItems e delivery dentro Order), riducendo il numero di query necessarie. I riferimenti (ref) sono usati per dati condivisi tra più documenti (utenti, ristoranti, piatti, ingredienti), mantenendo la consistenza senza duplicazione. Mongoose offre inoltre validazione dello schema a livello di modello, middleware per logica before/after save, e supporto nativo per ObjectId.

OpenStreetMap API. Per il calcolo delle distanze e dei costi di consegna, ho integrato le API di OpenStreetMap. Nominatim è usato per il geocoding degli indirizzi (conversione da testo a coordinate lat/lng), mentre OSRM è usato per il calcolo della distanza stradale tra due punti. Il flusso è il seguente: quando un cliente inserisce un indirizzo di consegna, il backend geocodifica l'indirizzo e calcola la distanza dalla filiale. Se la distanza supera 15km, l'ordine è rifiutato (il servizio di consegna è limitato a un raggio ragionevole). Il costo di consegna è calcolato come tariffa base + tariffa per km. Le coordinate del ristorante sono memorizzate nel database dopo la prima geocodifica, per evitare chiamate ripetute al servizio esterno. Questo approccio è gratuito e open source, a differenza di servizi come Google Maps che richiedono una chiave API e hanno limiti di utilizzo a pagamento.

Gestione stati ordine. Ho modellato gli stati degli ordini come un campo status con enum su stringa, piuttosto che come booleani separati o come un sistema di flags. I valori ammessi sono draft, ordered, preparing, ready, on_delivery, delivered. Il flusso è vincolato a una state machine: ogni transizione è validata dal backend prima di essere applicata. Per gli ordini pickup, la sequenza è ordered → preparing → ready → delivered. Per gli ordini delivery, è ordered → preparing → on_delivery → delivered. La validazione è enforce a livello di controller, non solo a livello di schema MongoDB, per impedire transizioni non ammesse (es. saltare da ordered a delivered senza passare per preparing). Il carrello in bozza è un ordine con status "draft" nella stessa collezione orders, per evitare di creare una tabella separata e mantenere la coerenza del modello dati.

Validazione input. La validazione e' implementata a due livelli: frontend e backend. A livello frontend, i form usano validazione HTML5 (required, type, minlength) e, dove necessario, validazione JavaScript custom prima di inviare la richiesta. A livello backend, ogni endpoint ha uno schema Joi di validazione definito nei moduli validations/. Questi schemi verificano il tipo, la lunghezza minima/massima, i valori ammessi (enum) e la presenza dei campi richiesti. I middleware validateRequest e validateQuery applicano gli schemi rispettivamente al body e ai parametri di query, restituendo un errore 400 con messaggio descrittivo in caso di validazione fallita. Questo doppio strato di validazione protegge sia dall'uso accidentale che da tentativi malintenzionati di inviare dati non validi al server.

10. SCENARI DI DEMO

Di seguito sono descritti cinque scenari end-to-end che ho preparato per la presentazione del progetto.

Scenario 1: Registrazione nuovo cliente

1. Aprire il browser e accedere all'URL del frontend (es. `http://localhost:4173`).
2. Cliccare sul bottone "accedi al terminale" per andare alla pagina di login.
3. Cliccare su "Registrati" per passare alla modalita' registrazione.
4. Compilare il form con nome, cognome, email, password e indirizzo.
5. Selezionare eventualmente alcune preferenze alimentari (vegetariano, senza glutine, ecc.).
6. Cliccare su "Registrati".
7. Verificare il redirect alla home page e la comparsa del nome utente nell'header.
8. Screenshot attesi: `screenshot-01-homepage.png`, `screenshot-02-register.png`, `screenshot-03-profile-after-register.png`.

Scenario 2: Cliente ordina con ritiro in sede

1. Dalla home page, cliccare su "apri il menu" per andare alla selezione ristorante.
2. Selezionare un ristorante dalla lista.
3. Navigare al menu del ristorante e aggiungere alcuni piatti al carrello (specificando le quantita').
4. Andare alla pagina di pagamento e scegliere "ritiro in sede".
5. Confermare l'ordine.
6. Verificare la creazione dell'ordine con codice alfanumerico e stato "ordered".
7. Andare alla pagina del profilo e verificare che l'ordine appaia nello storico.
8. Screenshot attesi: `screenshot-04-ristorante-list.png`, `screenshot-05-menu.png`, `screenshot-06-cart.png`, `screenshot-07-order-confirmed.png`, `screenshot-08-order-history.png`.

Scenario 3: Cliente ordina con consegna a domicilio (calcolo distanza)

1. Seguire i passaggi 1-3 dello scenario 2 per aggiungere piatti al carrello.
2. Nella pagina di pagamento, scegliere "consegna a domicilio".
3. Inserire l'indirizzo di consegna (es. "Via Torino 5, Milano").
4. Ottenere la stima della distanza e del costo di consegna.
5. Confermare l'ordine.
6. Verificare che l'ordine contenga i campi delivery con distanceKm e deliveryFee popolati.
7. Simulare il flusso di stato: il manager avanza l'ordine a preparing, poi on_delivery.
8. Il cliente conferma la ricezione e l'ordine passa a delivered.
9. Screenshot attesi: screenshot-09-checkout-delivery.png, screenshot-10-delivery-estimate.png, screenshot-11-order-tracking.png.

Scenario 4: Ristoratore aggiunge nuovo piatto

1. Effettuare il login con un account manager approvato con filiale.
2. Navigare alla dashboard manager e andare alla pagina Menu.
3. Cliccare su "Aggiungi piatto custom".
4. Compilare il form con nome, tipologia, prezzo, url foto e selezionare gli ingredienti.
5. Salvare il piatto.
6. Verificare che il piatto appaia nel menu della filiale e che sia possibile vederlo nella pagina pubblica del ristorante.
7. Screenshot attesi: screenshot-12-manager-login.png, screenshot-13-manager-menu.png, screenshot-14-add-dish-form.png, screenshot-15-dish-list.png.

Scenario 5: Cliente conferma ricezione ordine

1. Effettuare il login come cliente con un ordine a domicilio nello stato "on_delivery".
2. Andare alla pagina dettaglio ordine.
3. Cliccare sul bottone "Conferma ricezione".
4. Verificare che l'ordine passi allo stato "delivered" e che il timestamp di conferma sia registrato.
5. Verificare che l'ordine appaia nello storico come ordine passato.
6. Screenshot attesi: screenshot-16-order-on-delivery.png, screenshot-17-confirm-delivery.png, screenshot-18-order-delivered.png.

11. CONCLUSIONI

Il progetto Burger.sh e' stato un'esperienza formativa significativa che mi ha permesso di approfondire molteplici aspetti dello sviluppo web moderno. Ho imparato a progettare e implementare un'API REST completa con Express e MongoDB, gestendo autenticazione JWT, validazione input, paginazione e gestione errori in modo strutturato. A livello frontend, ho acquisito familiarità con Preact e il pattern di state management basato su context, che si e' rivelato efficace per un'applicazione SPA di medie dimensioni senza la complessità aggiuntiva di librerie esterne per lo stato.

Le difficoltà principali che ho incontrato sono state tre. La prima e' stata l'integrazione con le API di OpenStreetMap: il geocoding con Nominatim richiede una corretta gestione degli errori (indirizzi non trovati, rate limiting) e il caching delle coordinate per evitare chiamate ripetute. La seconda e' stata la gestione dei token JWT: ho dovuto progettare un sistema che gestisse correttamente la scadenza del token, il refresh implicito al login e il logout, con redirect appropriati in base al contesto. La terza e' stata la modellazione del flusso degli ordini: la state machine con transizioni vincolate e la distinzione tra carrello in bozza e ordine confermato richiede una progettazione attenta per evitare stati inconsistenti.

Ho risolto queste difficoltà attraverso iterazioni successive e testing. Per OpenStreetMap, ho creato un servizio `deliveryService` che gestisce il caching delle coordinate e i timeout. Per i token, ho implementato un middleware `authMiddleware` che verifica la firma JWT e un wrapper `fetchWithAuth` che gestisce automaticamente il 401. Per gli stati ordine, ho definito schemi di validazione rigorosi e testato tutte le transizioni possibili.

Come miglioramenti futuri, vorrei aggiungere un sistema di pagamento reale (integrazione con Stripe o simile), notifiche email per gli aggiornamenti degli ordini, supporto PWA per l'installazione su dispositivo mobile, e una dashboard admin più ricca con grafici e export dati. Inoltre, sarebbe utile implementare un sistema di rating/recensione per i ristoranti e un chatbot per assistere gli utenti nella navigazione.

12. BIBLIOGRAFIA E SITOGRAFIA

- Documentazione ufficiale Node.js: <https://nodejs.org/docs/latest/api>
- Documentazione MongoDB: <https://mongodb.com/docs/manual/tutorial>
- Documentazione Mongoose: <https://mongoosejs.com/docs/>
- Documentazione Express.js: <https://expressjs.com/>
- Documentazione Preact: <https://preactjs.com/guide/v10/>
- Documentazione Vite: <https://vitejs.dev/guide/>
- Documentazione Tailwind CSS: <https://tailwindcss.com/docs/>
- MDN HTML/CSS/JS: <https://developer.mozilla.org/en-US/docs/Web>
- OpenStreetMap API (Nominatim): <https://wiki.openstreetmap.org/wiki/Nominatim>
- OpenStreetMap API (OSRM): <https://project-osrm.org/>
- Swagger/OpenAPI: <https://swagger.io/>
- JWT.io Documentation: <https://jwt.io/introduction/>
- Joi Validation: <https://joi.dev/api/>
- bcrypt documentation: <https://www.npmjs.com/package/bcrypt>
- Helmet (HTTP security headers): <https://helmetjs.github.io/>
- htm (HTML templating): <https://github.com/developit/htm>

APPENDICE: CONFRONTO DOCUMENTAZIONE VS IMPLEMENTAZIONE REALE

Di seguito e' riportata una analisi delle discrepanze tra la documentazione originale (docs/ita/) e l'implementazione effettiva del progetto.

Frontend: tecnologia e architettura

La documentazione originale descriveva un frontend basato su HTML5, CSS3 e JavaScript vanilla con pagine separate. L'implementazione reale usa Preact (libreria UI dichiarativa) con Vite come bundler, Tailwind CSS v4 per lo styling, e un router custom che trasforma l'applicazione in una SPA (Single Page Application). I file HTML non esistono più come entità separate: esiste un singolo index.html che funge da shell, mentre tutto il contenuto e' renderizzato dinamicamente dai componenti Preact.

Ruolo amministratore

La documentazione originale menzionava il ruolo Admin solo come "estensione proposta". Nell'implementazione, il ruolo admin e' pienamente integrato con router dedicati (/admin/users, /admin/branches, /admin/stats), controller, middleware di autorizzazione (requireAdmin) e pagine frontend complete.

Carrello vs Ordine diretto

La documentazione originale descriveva la creazione diretta di ordini (POST /orders). L'implementazione reale introduce una risorsa separata /cart per il carrello in bozza, con operazioni CRUD su /cart/items e un endpoint di conferma /cart/confirm. Questo permette al cliente di costruire l'ordine in modo iterativo senza creare documenti a permanenza sul database finché non e' pronto a confermare.

Stato del ordine

La documentazione originale elencava solo gli stati ordered, preparing, ready, on_delivery, delivered. L'implementazione reale include anche lo stato draft per rappresentare il carrello in bozza nella stessa collezione orders.

Creazione ristorante

La documentazione originale prevedeva un unico endpoint POST /restaurants per la creazione. L'implementazione distingue tra POST /restaurants (solo admin) e POST /restaurants/first (manager approvato che crea la prima filiale). Questo riflette il flusso di onboarding reais del sistema.

Ingredienti come collezione separata

La documentazione originale parlava genericamente di "ingredienti" senza specificare una collezione dedicata. L'implementazione reale ha una collezione ingredients indipendente, con endpoint CRUD e gestione degli allergeni, accessibile solo ai manager.

Metodi di pagamento

La documentazione originale menzionava "eventuali metodi di pagamento associati" come funzionalità opzionale. L'implementazione include una collezione paymentMethods completa con CRUD e gestione del metodo predefinito.

Validazione e middleware

La documentazione originale non dettagliava i meccanismi di validazione. L'implementazione usa Joi per la validazione degli schema con middleware dedicati (`validateRequest`, `validateQuery`, `validateObjectId`) e un error handler globale che normalizza tutti gli errori in un formato JSON uniforme.

Rate limiting

Le rotte di setup (`POST /setup`, `POST /setup/request-pin`) sono protette da rate limiting (max 5 richieste per 15 minuti per IP) per prevenire abusi durante la fase di configurazione iniziale.

Testing

Il backend include una suite di test Jest con MongoDB Memory Server per testare controller e route in isolamento. Il frontend include test con `@testing-library/preact` e Vitest. La documentazione originale non menzionava alcuna strategia di testing.

Swagger/OpenAPI

La documentazione API e' interamente definita in un file YAML (`backend/swagger/openapi.yaml`) conforme alla specifica OpenAPI 3.0.3. Il file e' stato aggiornato per riflettere l'implementazione reale, includendo tutti gli endpoint sopra descritti.

Documento generato per il corso di Programmazione Web e Mobile, aa. 2025/2026.